

Domain Field Guide

Nine ICF domains, and the spec traps common to all of them

August 2026

Contents

Domain Field Guide	1
The shared skeleton	1
1. In-memory key-value database	2
2. File hosting / storage service	2
3. Banking system	2
4. File system simulator	3
5. Package manager	3
6. Build system / task scheduler	3
7. Text editor	4
8. Web crawler / rate limiter	4
9. LLM inference engine	4
Spec traps that appear across every domain	4
The 15-minute drill (do this instead of solving all nine)	5
What actually transfers	5

Domain Field Guide

Eight ICF domains. **Read this, don't solve it.** The goal is that no opening screen feels unfamiliar — not that you've pre-solved anything.

The thesis of this guide is that all eight are the same problem. Read two or three and you'll see it; read all eight and you'll stop worrying about which one you get.

The shared skeleton

Every domain reduces to:

A container	the system: database, bank, filesystem, engine
An entity	the thing stored: record, account, file, request
An identity	how entities are looked up: key, account id, path
A quantity	what gets summed and ranked: value, balance, size, cost

And every level asks the same question in different costumes:

Level	The question	Always needs
1	Can you store, retrieve, remove, and handle the missing case?	dict of entity objects
2	Can you filter, aggregate, and rank with exact tie-breaks?	<code>sorted(key=lambda x: (-a, b))</code>
3	Can you add time without breaking levels 1-2?	<code>_at(timestamp, ...)</code> delegating to internals
4	Can you reconstruct or reorganize state?	append-only log, or a second index

If you can answer those four in the abstract, the domain is decoration.

1. In-memory key-value database

State: {key: {field: Record}}

L1 | set, get, delete, compare_and_set |
L2 | scan, scan_by_prefix, top_n_keys by field count |
L3 | `_at` variants; fields expire via TTL |
L4 | `rollback(timestamp)` — restore prior state |

The one trick: `str.startswith(prefix)` for scans; sort field names with plain `sorted()` since they're strings.

This is the most frequently reported domain. You have a full mock of it in `mock_oa_kv/`.

2. File hosting / storage service

State: {filename: File(size, owner, created_at, ttl)}

L1 | upload (error if exists), get (size or None), copy (error if source missing, overwrite dest) |
L2 | `search(prefix)` — top 10 by size descending, ties by filename |
L3 | `_at` variants with TTL; search returns only “alive” files |
L4 | `rollback(timestamp)` with TTLs recalculated |

The one trick: “top 10 by size desc, tie by name asc” is `sorted(files, key=lambda f: (-f.size, f.name))[:10]`. Get that pattern into your fingers — some version of it appears in nearly every domain.

This is CodeSignal's own published example, so it's the most representative single problem in existence. Worth reading the spec in the technical brief.

3. Banking system

State: {account_id: Account(balance, history=[])}

L1 | `create_account` (False if exists), deposit, transfer (False if insufficient funds or missing account) |
L2 | `top_spenders(n)` — rank by total outgoing, ties by account id |

L3 | `_at` variants; **scheduled/pending payments** that execute at a future timestamp, plus `cancel_payment` |

L4 | `merge_accounts(a, b)` — combine balances *and* histories, keeping both ids working |

The one trick: scheduled payments mean you can't just mutate on call — you keep a list of pending operations and apply those with `execute_at <= timestamp` whenever a query arrives. Watch for “a transfer to a nonexistent account fails and money must not vanish.”

Watch out: the merge at L4 is where a weak L1 hurts. If `Account` owns its own history, merging is a list concat; if history lives in a global side-structure keyed by id, it's surgery.

4. File system simulator

State: a tree — `{path: Node(is_dir, size, children)}`, or a flat dict keyed by full path (usually simpler under time pressure)

L1 | `mkdir, create_file, read, delete` |

L2 | `list_dir` sorted, `total_size(path)` recursively |

L3 | permissions or ownership layered on; `_at` variants |

L4 | symlinks or mount points — resolving a path may now redirect |

The one trick: store full paths as flat dict keys (`"/a/b/c.txt"`) and derive parent/child with `path.rsplit("/", 1)`. Building a real tree of node objects is more faithful but costs time you don't have.

Watch out: symlinks introduce cycles. If L4 mentions them, cap resolution depth or track visited paths.

5. Package manager

State: `{name: Package(version, deps=[])}`

L1 | `add_package, remove, get_version` |

L2 | `list_dependencies(name)` — transitive, sorted |

L3 | version constraints — install the highest version satisfying a rule |

L4 | conflict detection / resolution when two packages demand incompatible versions |

The one trick: transitive dependencies = iterative traversal with a seen set. A deque and a while loop; no recursion needed, no graph theory.

Watch out: circular dependencies. The seen set handles them — without it you hang, and a hung test looks like a wrong answer.

6. Build system / task scheduler

State: `{task_id: Task(deps=[], state, duration)}`

L1 | `add_task, get_state, remove` |

L2 | `list_by_state`, counts per state |

L3 | execution respecting dependency order; timestamps |

L4 | caching (skip unchanged tasks) or parallel execution across N slots |

The one trick: “run in dependency order” is a topological sort, but you almost never need the textbook algorithm — repeatedly pick any task whose dependencies are all complete, with an explicit tie-break. That's a while loop over a list.

Watch out: the tie-break rule when several tasks are simultaneously ready. It will be specified. Read it twice.

7. Text editor

State: text plus `undo_stack / redo_stack`

L1 | `insert(pos, s), delete(pos, n), get()` |
L2 | `find(substring), replace_all` |
L3 | `undo / redo` |
L4 | multi-cursor, or collaborative edits from two sources |

The one trick: undo/redo is two stacks. Every mutation pushes an inverse operation onto undo and clears redo; undo pops from one and pushes onto the other. Push *operations*, not full text snapshots, unless the text is tiny.

Watch out: this domain's L3 is undo rather than timestamps — the one common exception to the “L3 = time” rule. But it's the same underlying requirement: you kept a history, so you can go backwards.

8. Web crawler / rate limiter

State: `{url: Page(content, fetched_at)}` plus per-domain counters

L1 | `fetch(url), has_visited, get_links` |
L2 | count by domain, top-N most linked |
L3 | rate limiting — at most N requests per domain per time window |
L4 | politeness delays, retry with backoff, or priority queues per domain |

The one trick: a sliding-window rate limiter is a deque of timestamps per domain: drop entries older than `now - window` from the left, then check `len(dq) < limit`.

Watch out: “per domain” means you key by hostname, not full URL. Extract it with string splitting — `urlib.parse` is standard library and allowed, but splitting on `/` is faster to write.

9. LLM inference engine

State: `{request_id: Request(prompt_tokens, max_tokens, ttl)}` plus workers

L1 | `submit, get_cost, cancel` |
L2 | `total_cost, top_n_costly` |
L3 | `_at` variants; requests expire |
L4 | multiple workers with least-loaded assignment |

The one trick: `ceil(a / b)` as `-(-a // b)`. Cost formulas always involve a ceiling division somewhere.

Reported for the recent Fellows cohort. Full mock in `mock_oa_infer/`.

Spec traps that appear across every domain

These are the misreadings that cost candidates whole levels. None are algorithmic — all are “the spec said something slightly different than you assumed.”

1. Deferred vs. immediate state change. A reported sitting had a `promote(worker_id)` method where the raise had to apply on the worker's *next* `register()` call, not when `promote()` ran. Mutating immediately failed every subsequent test. When a method sounds like it changes state, ask *when* the change takes effect. Implement it as a flag the next operation consumes:

```
def promote(self, worker_id):
    self.workers[worker_id].pending_raise = True    # only sets a flag

def register(self, worker_id, timestamp, action):
    w = self.workers[worker_id]
    if action == "enter" and w.pending_raise:
        w.rate *= 1.10                               # applied here
        w.pending_raise = False
```

2. In-progress items excluded from totals. A time tracker's `get_total_time` had to sum only *closed* segments, ignoring an open one. The same shape appears as pending transfers, uncommitted writes, unassigned requests. Aggregations usually count completed things only — check.

3. Return type for the missing case. `None`, `0`, `False`, `[]`, or `raise`? Every domain has this and it's always in the tests. `get` on a missing key returning `None` while `count` returns `0` is normal and easy to conflate.

4. Ties, always. Any “top N” or “pick one” has a tie-break rule. Read whether ties break by name ascending, id ascending, insertion order, or highest id. Write it into the sort key explicitly rather than relying on stability.

5. Inclusive vs. exclusive boundaries. A TTL of 5 from time 1 usually means alive on `[1, 6)` — dead *at* 6, not after. Off-by-one here silently breaks half a level.

6. Overwrite vs. reject on duplicates. Does a second create with an existing id overwrite, return `False`, or `raise`? Domains differ, and the spec often states it in one clause you skim past.

7. Does an update reset the clock? Re-setting a field with a TTL usually restarts its lifespan from the new timestamp. Sometimes it doesn't. The tests will say.

The habit that catches all seven: after reading the spec, write a short list of every *decision point* you noticed — return types, tie-breaks, boundaries — before writing code. Two minutes, and it's the difference between a level scoring 100% and 40%.

The 15-minute drill (do this instead of solving all nine)

For **three** domains you haven't practiced — pick from banking, filesystem, package manager, text editor — spend five minutes each writing only:

1. The `@dataclass` for the entity: what fields, and which need `created_at` and `ttr`?
2. The `__init__` of the container: which dicts, which lists?
3. The signature list for Level 1's three or four methods.

Don't implement anything. You're training the first ten minutes of the assessment, which is the part that actually decides your score. If you can produce a sane skeleton for any domain in five minutes, you have what this guide is for.

What actually transfers

Across all nine domains, the entire technical toolkit is:

- dict of entity objects, occasionally nested two deep
- `sorted(..., key=lambda x: (-quantity, name))` with an explicit tie-break
- `str.startswith` for prefix filtering
- deque for queues and sliding windows
- set for “already seen”
- ceiling division
- an append-only list for history

That’s it. No algorithms, no clever data structures. If your solution needs something outside that list, re-read the problem — you’ve almost certainly overcomplicated it.